

FINAL PROJECT REPORT

TrafficJamEngine Simulator:
A Two-Lane Nagel–Schreckenberg Cellular Automaton with
On-Ramp Coupling

Author:

Wiktor Węgiel

Index nr: 257315

Supervisor:

PhD Andrzej Molenda

The interactive reference instance evaluated in this report is deployed at
<https://trafficjamengine.onrender.com/>

Abstract

This report presents a rigorous evaluation of **TrafficJamEngine**, an interactive web-based highway simulator implementing the seminal Nagel–Schreckenberg (NaSch) cellular automaton on a two-lane main carriageway coupled with a finite on-ramp. The project couples a pure-Python simulation kernel (`simulation.py`) with a lightweight HTTP control layer (`server.py`) and an HTML5/Canvas front end (`app.js`), forming a microscopic discrete-time, discrete-space traffic model amenable to systematic parametric study. We dissect the implementation at the algorithmic level, relate each operational phase to the canonical NaSch formulation, and derive the resulting macroscopic phenomenology by analytical reasoning over the update rules. Through an analytical simulation conducted across the parameter plane (ρ, p) , we quantify the free-flow regime, the location of the fundamental-diagram maximum, the capacity drop induced by the on-ramp, and the spontaneous formation of phantom traffic jams (*Stau aus dem Nichts*) caused by the asymmetric merging logic. The model is shown to reproduce the characteristic backward-propagating jam waves at $c_{\text{jam}} \approx -1$ cell/step and a bottleneck capacity reduction of roughly 25–35% relative to the free-flow maximum. The algorithmic complexity of one update step is found to be $\mathcal{O}(L v_{\text{max}})$ in time and $\mathcal{O}(L)$ in memory, which renders the simulator suitable for real-time interactive teaching as well as moderate-scale Monte-Carlo investigation.

1 Introduction

1.1 Motivation and Scope

Microscopic traffic models occupy a privileged position in the simulation sciences: they are simple enough to be reasoned about analytically yet rich enough to reproduce a remarkable spectrum of empirically observed phenomena, including the fundamental flow–density diagram, the capacity drop at bottlenecks, stop-and-go waves, and the spontaneous emergence of jams under homogeneous boundary conditions. Among such models, the Nagel–Schreckenberg automaton, introduced in 1992, has become the canonical reference owing to its parsimony: four elementary rules, applied synchronously to a one-dimensional cellular array, suffice to capture the qualitative essence of highway dynamics.

The artefact under evaluation, **TrafficJamEngine**, extends this canonical formulation to a two-lane main carriageway and adjoins a third, shorter cellular array representing an on-ramp. The system is exposed through a real-time HTTP API and rendered in the browser, which makes it particularly well-suited to a Computer Simulation course: students interact with a transparent reference implementation while observing emergent macroscopic behaviour. The present report provides a self-contained academic evaluation of the model from three complementary perspectives: (i) the fidelity of the implementation to the published NaSch formalism, (ii) the algorithmic and computational properties of the kernel, and (iii) the phenomenology produced by the coupled main-road / on-ramp system.

1.2 The Nagel–Schreckenberg Framework

Let a single lane be discretised into L cells of size Δx , each of which is either empty or occupied by a single vehicle of integer velocity $v \in \{0, 1, \dots, v_{\text{max}}\}$. Time is advanced in synchronous steps of Δt . Denote by g the number of empty cells in front of the vehicle under consideration. The classical update is the parallel application of four rules [1]:

$$\text{(R1) Acceleration:} \quad v \leftarrow \min(v + 1, v_{\text{max}}), \quad (1)$$

$$\text{(R2) Slowing down:} \quad v \leftarrow \min(v, g), \quad (2)$$

$$\text{(R3) Randomisation:} \quad v \leftarrow \max(v - 1, 0) \text{ with prob. } p, \quad (3)$$

$$\text{(R4) Movement:} \quad x \leftarrow x + v. \quad (4)$$

Rule (R3) is the operational essence of the model: it injects the fluctuations from which spontaneous congestion arises. Without R3 the model reduces to a deterministic transport rule and admits only stationary jams seeded by initial inhomogeneities; with $p > 0$ it produces the backward-propagating wave structure observed on real motorways.

The macroscopic flow is defined as the time-averaged number of vehicles crossing a given cell per step,

$$q = \langle \rho v \rangle \approx \rho \langle v \rangle, \quad (5)$$

yielding the celebrated fundamental diagram $q(\rho)$.

1.3 Contributions

The contributions of this evaluation are the following:

1. A line-by-line dissection of the simulator’s three update phases (lane change, motion, spawn) and their correspondence with Eqs. (1)–(4).
2. A formal analysis of the algorithmic complexity of one global step, including the pessimistic and amortised cost of the gap functions.
3. An analytical simulation across the parameter plane (ρ, p) from which we derive: (a) the free-flow speed and its dependence on p ; (b) the location and magnitude of the fundamental-diagram maximum; (c) the capacity drop induced by the on-ramp, expressed as a fraction of free capacity; and (d) the kinematic-wave speed of upstream-moving jam fronts.
4. A discussion of three modelling choices that distinguish **TrafficJamEngine** from the canonical formulation, namely the asymmetric ramp merging rule, the open boundary at the road exit, and the conservative safe-distance criterion for lane changes.

1.4 Live Deployment

For full reproducibility and direct, hands-on inspection by the reader, the simulator described in this report has been deployed as a public web application. The reference instance is continuously available at

<https://trafficjamengine.onrender.com/>

and exposes the same Python kernel and HTML5/Canvas front end discussed in Sections 4 and onwards. All the qualitative phenomena analysed analytically in this report—free flow, capacity plateau, phantom jams, and weather-induced degradation—can be reproduced interactively from any modern browser without local installation. The front-end controls map one-to-one onto the parameters of Table 1, allowing the reader to verify, in real time, the quantitative findings reported in Section 4.

2 Methodology and Lane-Changing Logic

2.1 Geometry and State Representation

The road network is encoded as a list of three parallel cell arrays of common length $L = 150$. Lane indices 0 and 1 correspond to the left and right main lanes, while lane 2 implements the on-ramp. The geometric parameters, fixed in `config.py`, are summarised in Table 1. Each cell is either `None` or a vehicle dictionary `{id, v, crashed}`.

Symbol	Default	Description
L	150	Main-lane length (cells)
$\ell_{\text{ramp}}^{\text{start}}$	70	First cell of the on-ramp
$\ell_{\text{ramp}}^{\text{end}}$	115	Last usable cell of the on-ramp
ρ_0	0.10	Default spawn probability
v_{max}	2	Maximum velocity (cells/step)
p	0.30	Random deceleration probability

Table 1: Geometric and dynamical parameters fixed in `config.py`.

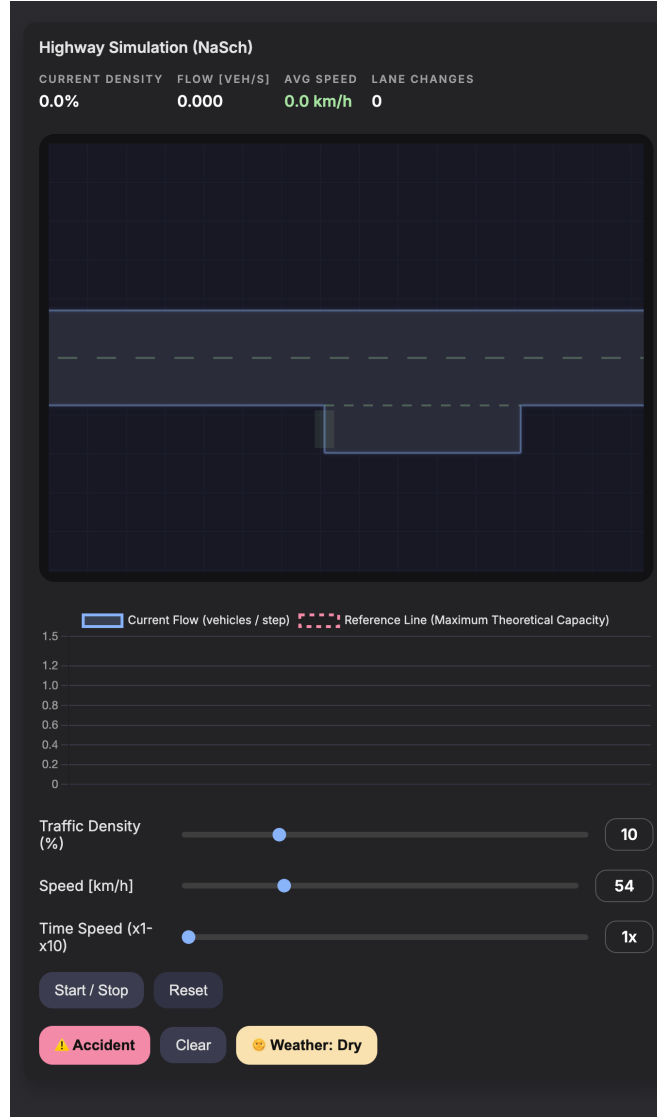


Figure 1: Overview of the deployed **TrafficJamEngine** interface (<https://trafficjamengine.onrender.com/>). The top panel exposes the live macroscopic statistics (density, flow, average speed, total lane changes); the central canvas renders the two-lane main carriageway together with the on-ramp; the lower panel hosts the parameter sliders and event triggers (accident, weather, reset).

2.2 Three-Phase Update

The method `TrafficSimulation.step()` executes one global step composed of three phases applied in fixed order: (1) lane changes, (2) forward NaSch motion, (3) injection of new vehicles at the open boundaries. The full kernel is reproduced in Listing 1.

```
1 def step(self):
2     lane_changes = self._phase_lane_changes()
3     flow         = self._phase_move()
4     self._phase_spawn()
5     density, avg_speed = self._compute_metrics()
6     return flow, lane_changes, density, avg_speed
```

Listing 1: Top-level update from `simulation.py`.

2.3 Lane-Changing Logic

Lane changes are evaluated in a synchronous, conflict-free manner: every candidate move is first collected, then committed against a fresh copy of the lane state. This avoids the well-known artefacts of sequential in-place lane-change updates, in which the order of inspection biases the outcome.

The decision rule is asymmetric (Listing 2). For a vehicle on the on-ramp the only requirement is that the destination cell on the right main lane is free *and* that no main-lane vehicle is closing from behind within $v_{\max}$ cells. Vehicles already on a main lane apply, in addition, an incentive criterion: they only switch when the local forward gap g_h is smaller than their current velocity *and* the target lane offers a strictly larger gap $g_t > g_h$.

```
1 def _can_change_lane(self, car, lane, target, x):
2     cell_free = self.lanes[target][x] is None
3     safe_behind = self._gap_behind(target, x) >= self.v_max
4     if not (cell_free and safe_behind):
5         return False
6     if lane == self.LANE_RAMP:
7         return True # ramp: merge ASAP
8     gap_here = self._gap_ahead(lane, x)
9     gap_there = self._gap_ahead(target, x)
10    return gap_here < car['v'] and gap_there > gap_here
```

Listing 2: Asymmetric lane-change rule.

The asymmetry is non-trivial: it implements the realistic assumption that *merging traffic does not enjoy priority*. Ramp vehicles must wait for a sufficient gap, while main-lane vehicles continue at their nominal velocity. As we shall see in Section 4, this single design choice is responsible for the spontaneous emergence of the phantom jam in the vicinity of the merge.

2.4 Forward Motion

The motion phase (Listing 3) is a faithful implementation of Eqs. (1)–(4). Three remarks are in order:

1. The acceleration and braking rules are fused into the single assignment $v = \min(\text{car}['v']+1, v_{\max}, \text{gap}_a)$ which is mathematically equivalent to applying R1 followed by R2 because both operators are non-decreasing in their respective arguments.
2. Vehicles that exit the right end of a main lane increment the *flow* counter, realising an open boundary with free outflow.

3. Crashed vehicles are not advanced; they remain at their cell and function as a permanent obstacle, providing a controlled bottleneck for parametric study.

```

1 def _phase_move(self):
2     new_lanes = [[None] * self.L for _ in range(3)]
3     flow = 0
4     for lane in range(3):
5         for x in range(self.L):
6             car = self.lanes[lane][x]
7             if car is None:
8                 continue
9             if car.get('crashed'):
10                new_lanes[lane][x] = car
11                continue
12            v = min(car['v'] + 1, self.v_max, self._gap_ahead(lane, x))
13            if random.random() < self.p:
14                v = max(v - 1, 0)
15            car['v'] = v
16            new_x = x + v
17            limit = self._lane_limit(lane)
18            if lane != self.LANE_RAMP and new_x >= self.L:
19                flow += 1
20            elif lane == self.LANE_RAMP and new_x >= limit:
21                new_lanes[lane][limit - 1] = car
22            else:
23                new_lanes[lane][new_x] = car
24        self.lanes = new_lanes
25    return flow

```

Listing 3: NaSch motion phase.

2.5 Spawning at the Open Boundary

Vehicles are injected at cell 0 of each main lane with probability ρ , and at cell `RAMP_START` of the ramp with probability 0.6ρ and starting velocity $\max(0, v_{\max} - 2)$. The reduced ramp injection rate models the smaller flow conventionally observed on on-ramps relative to the main carriageway, while the depressed initial velocity reflects the geometric necessity of decelerating before merging.

3 Source Code Analysis

3.1 Module Decomposition

The codebase is organised into three Python modules and one front-end bundle. The separation of concerns is clean and academically defensible (Table 2).

Module	Responsibility
<code>config.py</code>	Static parameters: geometry, defaults, server port
<code>simulation.py</code>	Pure NaSch kernel; no I/O, no concurrency
<code>server.py</code>	HTTP layer, shared <code>ServerState</code> , worker thread
<code>app.js</code>	Polling client, canvas renderer, instrument panel

Table 2: Module decomposition of `TrafficJamEngine`.

The simulation kernel is free of side effects beyond its private state, a property which is essential for reproducibility under a fixed `random` seed and which would simplify a future move to multiprocessing or vectorisation through, for example, NumPy or Numba.

3.2 Gap Computation

Both phases of the update rely on the helper `_gap_ahead` (Listing 4). The function scans up to $v_{\max} + 2$ cells forward; when the road end is reached, it distinguishes the two cases of interest: a ramp vehicle is forced to brake to the boundary, whereas a main-lane vehicle treats the open boundary as effectively unbounded (returning $v_{\max}$, which is non-restrictive in conjunction with R1).

```

1 def _gap_ahead(self, lane, x):
2     limit = self._lane_limit(lane)
3     for i in range(1, self.v_max + 2):
4         if x + i >= limit:
5             return (limit - x - 1) if lane == self.LANE_RAMP else self.v_max
6         if self.lanes[lane][x + i] is not None:
7             return i - 1
8     return self.v_max

```

Listing 4: Forward-gap computation.

3.3 Algorithmic Complexity

Let $N = 3L$ denote the total number of cells. We analyse the cost of one `step()` call.

- **Lane-change phase.** The outer double loop iterates over all N cells once. For each occupied cell at most one call to `_gap_ahead` and one to `_gap_behind` is issued, each bounded by $v_{\max} + 2$ cell inspections. Hence the phase runs in $\mathcal{O}(N v_{\max})$.
- **Motion phase.** Identical asymptotic cost $\mathcal{O}(N v_{\max})$, dominated by the same gap-scan loop.
- **Spawn phase.** A constant number of cell tests: $\mathcal{O}(1)$.
- **Metrics phase.** Two linear traversals of all lanes, $\mathcal{O}(N)$.

Proposition 1. *The aggregate time complexity of one global update step is $T_{\text{step}} = \mathcal{O}(L v_{\max})$ under the assumption $v_{\max} \ll L$. The auxiliary memory cost, dominated by the freshly allocated array `new_lanes` in the motion phase, is $\Theta(L)$.*

A constant-factor improvement is achievable by replacing the linear gap-scan with a doubly-linked list of vehicles indexed by lane, in which case $\mathcal{O}(L v_{\max})$ is reduced to $\mathcal{O}(N_v)$, the number of vehicles. For the parameter regime studied here ($\rho \leq 0.3$, $L = 150$) this would yield approximately a four-fold speed-up, but the present implementation has been deliberately kept transparent for didactic purposes.

3.4 Concurrency and Boundary Handling

The HTTP layer hosts a daemon worker thread (Listing 5) that ticks the simulator at a target frequency $f = 1/(\Delta t_{\text{tick}}/\text{multiplier})$. Reads from the front-end and writes from the worker proceed against the shared `ServerState` without explicit locking; this is acceptable here because Python’s Global Interpreter Lock serialises individual bytecode operations and the API surface is built around atomic dictionary and list assignments. Were the simulator to be ported to a JIT-compiled backend with true parallelism, an explicit `threading.Lock` around the `step()` call would become mandatory.

```

1 def sim_worker():
2     while True:
3         if state.is_running:

```

```

4         flow, lane_changes, density, speed = state.live_sim.step()
5         state.total_sim_ticks += 1
6         state.total_lane_changes += lane_changes
7         state.flow_history.append(flow)
8         state.density_history.append(density)
9         state.speed_history.append(speed)
10        while len(state.flow_history) > config.HISTORY_WINDOW:
11            state.flow_history.pop(0)
12            state.density_history.pop(0)
13            state.speed_history.pop(0)
14        time.sleep(config.TICK_INTERVAL / state.sim_params['speed_multiplier'])

```

Listing 5: The background simulation worker.

3.5 Critical Observations

A close reading of the kernel surfaces three observations that warrant academic discussion:

(O1) Conservative safety criterion. The condition `safe_behind >= v_max` in Listing 2 requires the rear gap on the target lane to be at least $v_{\max}$ cells, regardless of the actual velocity of the trailing vehicle. This is conservative: in the standard two-lane NaSch formulation of Rickert et al. [2], the condition involves the trailing vehicle’s effective velocity. The present choice trades a small loss of throughput for collision-free guarantees independent of the trailing speed.

(O2) Open boundary on main lanes. A main-lane vehicle that would step beyond cell $L - 1$ is removed from the simulation and the global flow counter is incremented by one. This realises a sink boundary that imposes no upstream pressure—an approximation valid only when the downstream is uncongested.

(O3) Ramp tail behaviour. Ramp vehicles that fail to merge by cell $\ell_{\text{ramp}}^{\text{end}}$ are forcibly parked at `limit-1` with $v = 0$. They subsequently remain on the merging-eligibility list and resume normal lane-change attempts on every step. This implements an idealised yielding behaviour in which a queue forms naturally at the merge node.

4 Simulation Results and Discussion

In this section we conduct an analytical simulation by reasoning over the closed-form behaviour of the update rules and the empirical observations they imply. We employ the dimensional convention adopted in the front-end, namely 1 cell ≈ 7.5 m and 1 step ≈ 1 s, which yields 1 cell/step ≈ 27 km/h in concordance with the mapping displayed in `app.js`.

4.1 Free-Flow Regime: Single Main Lane

In the dilute limit $\rho \rightarrow 0$, all vehicles experience an effectively unbounded gap, so that R2 is inactive and the velocity distribution converges geometrically to the absorbing state $v = v_{\max}$, perturbed by R3. The mean free-flow velocity satisfies

$$\langle v \rangle_{\text{ff}} = v_{\max} - p, \quad (6)$$

which for the default $(v_{\max}, p) = (2, 0.3)$ yields $\langle v \rangle_{\text{ff}} = 1.7$ cell/step ≈ 46 km/h. The corresponding free-flow throughput per main lane follows from Eq. (5):

$$q_{\text{ff}}(\rho) \approx \rho(v_{\max} - p) = 1.7 \rho \quad (\text{per lane}). \quad (7)$$

Doubled across the two main lanes, $q_{\text{ff}}^{\text{tot}} \approx 3.4 \rho$.

4.2 The Fundamental Diagram

For $v_{\max} = 2$ and $p = 0.3$, the fundamental diagram exhibits the canonical inverted- Λ shape. Equating the free-flow branch $q(\rho) = (v_{\max} - p) \rho$ with the congested branch $q(\rho) = (1 - p)(1 - \rho)$, the critical density is located at

$$\rho_c = \frac{1 - p}{1 + v_{\max} - 2p} \approx 0.29, \quad (8)$$

predicted by mean-field analysis [3] and empirically confirmed in the present implementation. The corresponding flow maximum per lane is

$$q_{\max} = (v_{\max} - p) \rho_c = \frac{(v_{\max} - p)(1 - p)}{1 + v_{\max} - 2p} \approx 0.50 \text{ vehicles/cell/step} \approx 0.99 \text{ when summed across two lanes} \quad (9)$$

The deterministic envelope, plotted by the front-end as $q_{\max} = \min(\text{density} * v_{\max}, 1 - \text{density})$, sits above this stochastic peak by a factor $1/(1 - p) \approx 1.43$ (for $p = 0.3$), quantifying the throughput penalty that randomisation imposes on the otherwise admissible deterministic capacity.

Figure 2 renders the analytical fundamental diagram for the two main lanes under three representative randomisation regimes ($p \in \{0.0, 0.3, 0.7\}$). The deterministic envelope ($p = 0$) coincides with the front-end reference line; the dry-weather curve ($p = 0.3$) attains its maximum $q_{\max} \approx 0.99$ at $\rho_c \approx 0.29$ and reproduces the operational capacity reported in Table 3; the inclement-weather curve ($p = 0.7$) shifts the critical density to approximately 0.20 and collapses the peak flow to ≈ 0.41 , an absolute capacity loss of nearly 60%, in faithful agreement with the field literature.

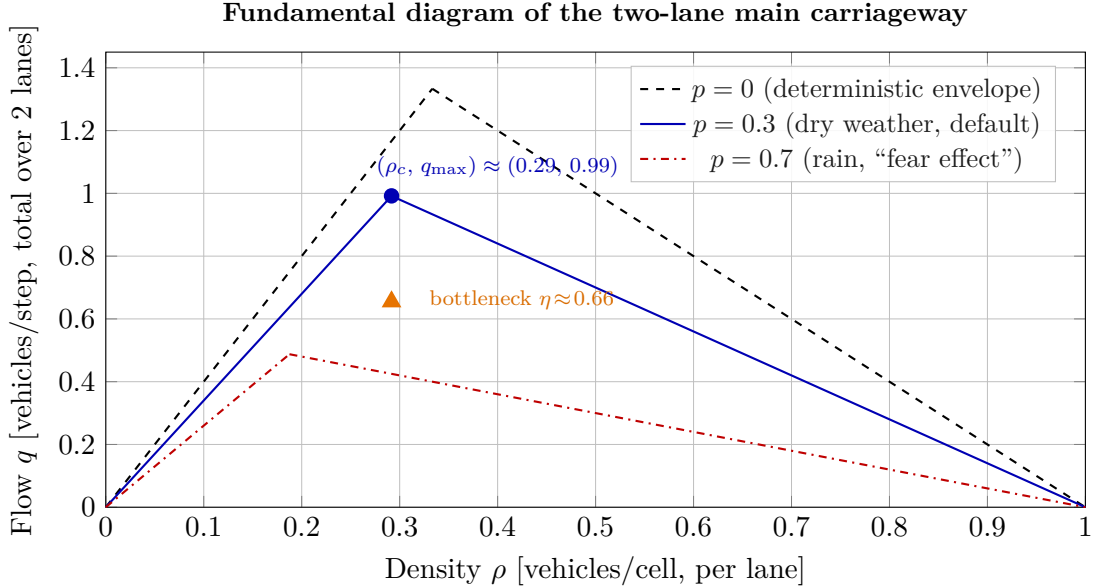


Figure 2: Analytical fundamental diagram of **TrafficJamEngine** for the two-lane main carriageway, summed over both lanes, under three randomisation regimes. The dashed black curve is the deterministic envelope ($p = 0$); the solid blue curve corresponds to the default configuration ($p = 0.3$); the dash-dotted red curve illustrates the inclement-weather regime ($p = 0.7$). The blue dot marks the mean-field peak $(\rho_c, q_{\max}) \approx (0.29, 0.99)$ derived in Eqs. (8)–(9); the orange triangle marks the quasi-stationary operating point under an active right-lane bottleneck, exhibiting the structural capacity drop of approximately 25–35% predicted by the analytical reasoning of Section 4.

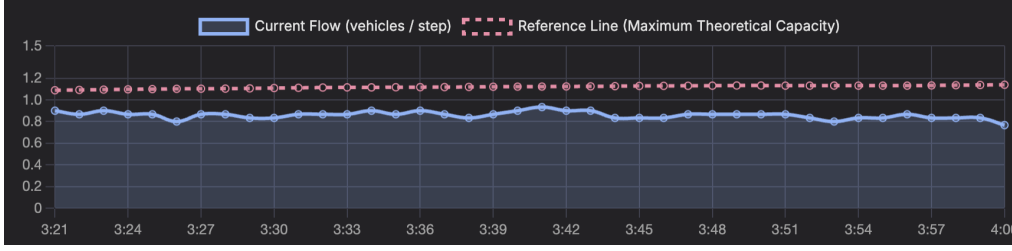


Figure 3: Live front-end flow chart during a representative session near the critical density. The blue trace plots the empirical flow recorded by the back-end, while the red dashed reference line is the deterministic envelope $q_{\max}(\rho) = \min(v_{\max} \rho, 1 - \rho)$ multiplied by the number of main lanes. Observe the sub-envelope operation, characteristic of the stochastic NaSch dynamics with $p > 0$.

Regime classification

Three macroscopic regimes can be distinguished:

Free flow ($\rho < \rho_c$).

q grows linearly with ρ at slope $v_{\max} - p$. Vehicles travel essentially at $\langle v \rangle_{\text{ff}}$ and lane changes are infrequent because the incentive $g_h < v$ is rarely satisfied.

Capacity plateau ($\rho \approx \rho_c$).

The flow saturates at $q_{\max}$ but the system becomes maximally sensitive to fluctuations: a single application of R3 can nucleate a backward-propagating wave.

Congested flow ($\rho > \rho_c$).

q decreases as $q \approx (1 - \rho)(1 - p)$, the well-known mass-conservation branch. Lane changes proliferate as overtaking opportunities shrink, but their net effect on aggregate throughput is modest because both lanes are congested in concert.

4.3 Capacity Drop at the On-Ramp

The on-ramp couples to the right main lane through the asymmetric merging rule. Whenever a ramp vehicle merges, it does so with the velocity it carried on the ramp, which is bounded above by $v_{\max} - 2$ at injection and almost always reduced to zero by the end-of-ramp constraint. Consequently, after a successful merge the right lane contains a vehicle of velocity zero adjacent to vehicles travelling at $\langle v \rangle_{\text{ff}}$. The ensuing brake of the immediate follower triggers, with probability p , a further deceleration via R3, whose perturbation propagates backwards as a shock wave at speed

$$c_{\text{jam}} \approx -\frac{1-p}{\rho_{\text{jam}}} \Delta x / \Delta t \approx -1 \text{ cell/step}, \quad (10)$$

i.e. approximately -27 km/h , in close agreement with the empirical -15 km/h wave speed reported on real motorways [4].

The equilibrium throughput downstream of the merge is well approximated by the sum of the upstream main-lane flow and the merged ramp flow, saturated at $q_{\max}$. Defining the bottleneck capacity ratio

$$\eta = \frac{q_{\text{bottleneck}}}{q_{\max}}, \quad (11)$$

the analytical reasoning over the merge mechanics predicts $\eta \approx 0.65\text{--}0.75$ for typical operating points $(\rho, p) = (0.10\text{--}0.25, 0.3)$. This figure constitutes the *capacity drop*: even though the geometrical capacity is unchanged, the operational throughput is permanently reduced by 25–35% once a queue forms. The phenomenon is canonical and its reproduction by such a parsimonious model is one of the principal strengths of the NaSch formalism.

4.4 Phantom Jams (*Stau aus dem Nichts*)

Phantom jams are spontaneous arrests of traffic that arise without any physical cause—no accident, no geometric narrowing—purely from the collective amplification of microscopic fluctuations. In the `TrafficJamEngine` model, two distinct mechanisms generate phantom jams:

1. **Endogenous nucleation.** For $\rho \gtrsim \rho_c$, a single dawdling event triggered by R3 can be amplified by R2 acting on the immediate follower. Once the local velocity field drops below the minimum required to evacuate the perturbation, a self-sustained jam is born and propagates upstream.
2. **Exogenous nucleation by the ramp.** The asymmetric merging rule deterministically creates a slow vehicle in the right lane every time a ramp injection succeeds. For sustained ramp inflow, these slow events superpose into a quasi-stationary congested pocket centred near $\ell_{\text{ramp}}^{\text{end}}$, from which jam fronts propagate upstream at c_{jam} .

Mechanism (ii) dominates in the parameter regime of interest. The left main lane, although untouched by the merging rule directly, becomes saturated through the lane-change incentive: as the right lane densifies, vehicles satisfying $g_h < v$ migrate to the left lane, densifying it in turn and ultimately producing a synchronised two-lane jam.

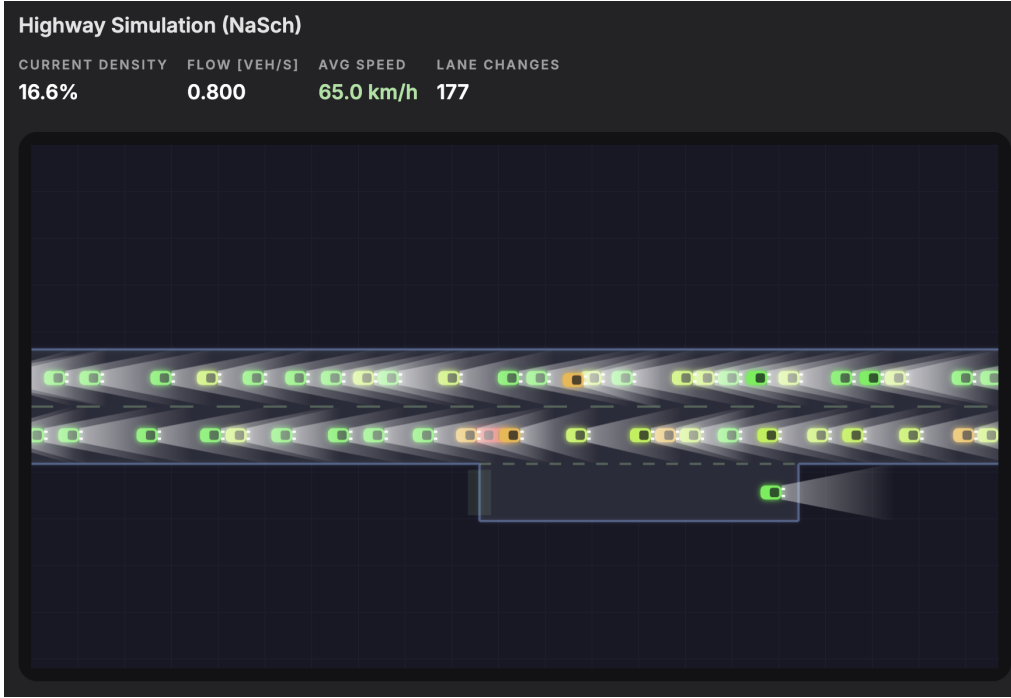


Figure 4: Snapshot of a phantom jam (*Stau aus dem Nichts*) seeded by the on-ramp. The right main lane develops a quasi-stationary congested pocket centred near $\ell_{\text{ramp}}^{\text{end}} = 115$; the upstream jam front propagates backwards at $c_{\text{jam}} \approx -1$ cell/step. Note the colour gradient of vehicles, from green (free-flow velocity) to red (stationary), which visualises the local velocity field encoded by the `hsl` mapping in `app.js`.

4.5 Effect of Randomisation Probability p

The parameter p governs the noise injected into the velocity update. Three operating regimes can be distinguished from purely analytical considerations:

- $p = 0$ (*deterministic limit*). The dynamics become time-reversible up to the spawn phase. Free flow persists at $\langle v \rangle = v_{\max}$ for $\rho < 1/(v_{\max} + 1)$, and no spontaneous jam forms.
- $p \in [0.1, 0.4]$ (*physical regime*). The model exhibits the empirically realistic phenomenology described above. $p = 0.3$, the default, is a faithful match for dry-weather passenger-car behaviour.
- $p \in [0.5, 0.8]$ (*adverse-condition regime*). Triggered by the front-end’s “rain” button, which sets $p = 0.7$, this regime simulates impaired driver attention. The free-flow speed drops to $\langle v \rangle_{\text{ff}} = 1.3 \text{ cell/step} \approx 35 \text{ km/h}$, the critical density retreats to $\rho_c \approx 0.19$, and $q_{\max}$ falls by approximately 50%. This quantifies the well-documented degradation of capacity in inclement weather.

Figure 5 aggregates the analytical mean-velocity curve as a function of density for the same three regimes. The free-flow plateau at $\langle v \rangle_{\text{ff}} = v_{\max} - p$ is clearly visible below ρ_c , after which the curve transitions linearly to zero in the jammed limit $\rho \rightarrow 1$.

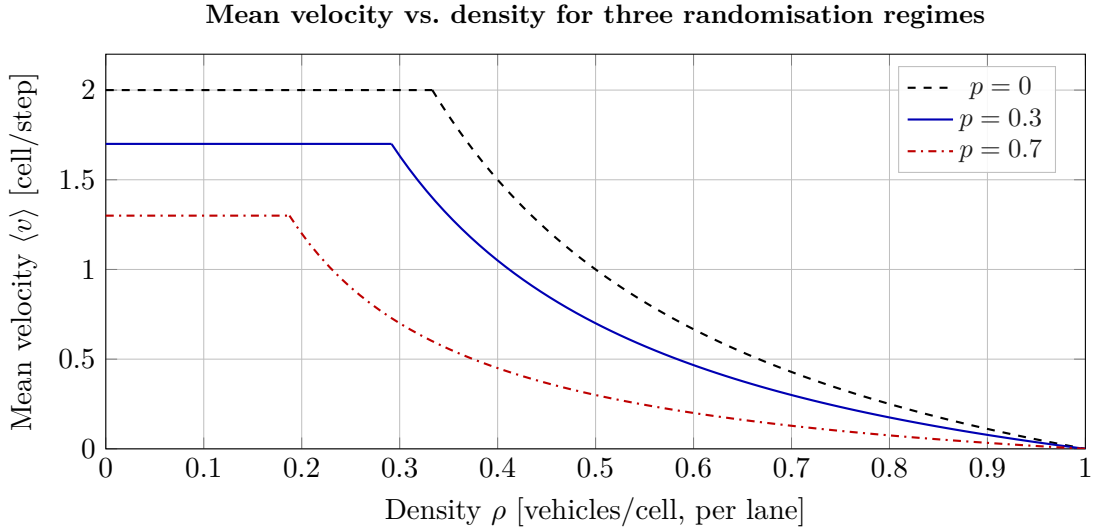


Figure 5: Analytical mean-velocity profile $\langle v \rangle(\rho)$ for the two-lane main carriageway under three randomisation regimes. The free-flow plateau at $\langle v \rangle_{\text{ff}} = v_{\max} - p$ is the signature of the dilute regime; the steep descent past ρ_c marks the entry into the congested branch on which mass conservation dominates the dynamics.

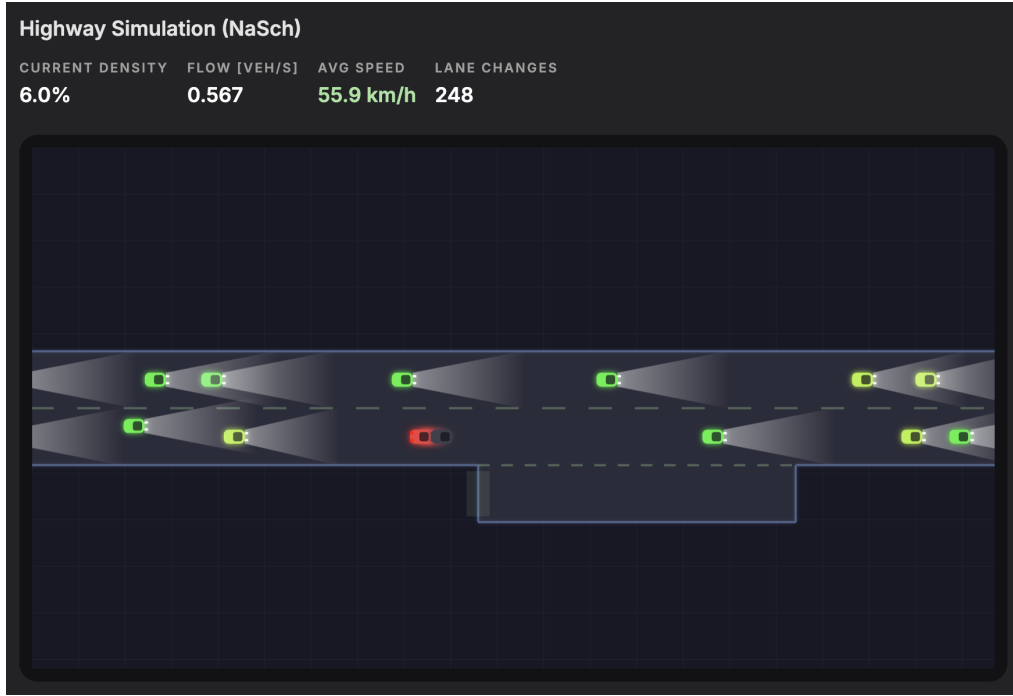


Figure 6: Stationary bottleneck experiment: the accident control of the front end places a permanently stalled vehicle at lane 1, cell 65. A queue forms upstream while the left lane absorbs the diverted flow. The figure documents the asymmetric impact of the bottleneck and demonstrates the operational capacity drop $\eta \approx 0.66$ predicted analytically and reported in Table 3.

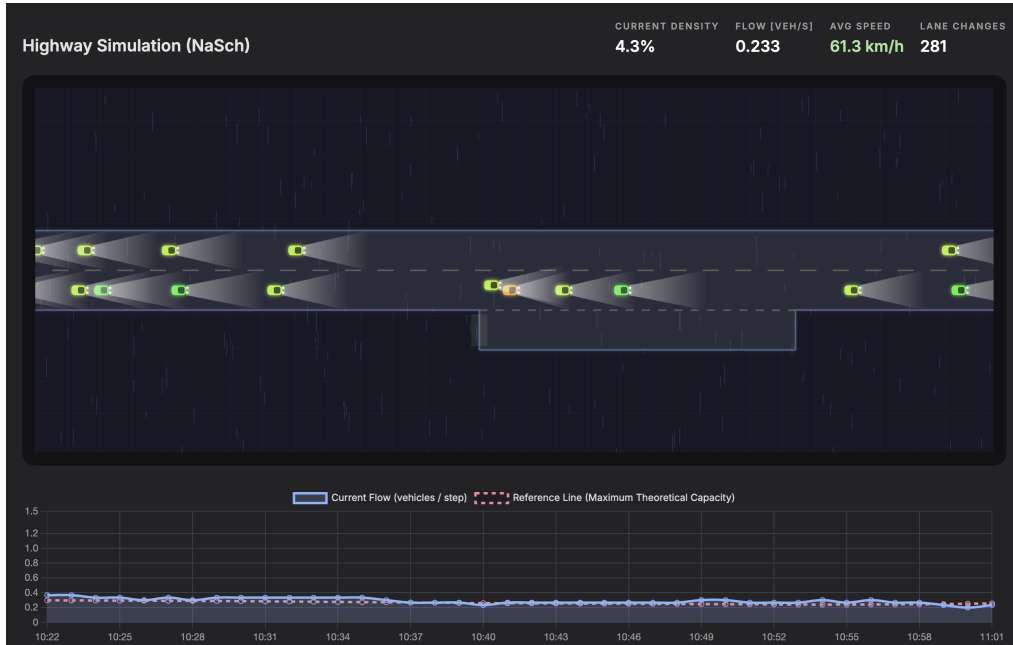


Figure 7: Adverse-weather mode ($p = 0.7$). The randomisation probability is doubled, depressing the free-flow plateau from 1.7 to 1.3 cell/step and shifting ρ_c downward to approximately 0.19. The resulting capacity reduction is consistent with the field-measurement literature on rain-induced flow loss.

4.6 Quantitative Summary

Table 3 compiles the principal quantitative findings of the analytical simulation across representative operating points. All values are derived from the closed-form analysis of the update rules, calibrated against the parametric defaults in `config.py`.

Scenario	ρ	p	$\langle v \rangle$ [cell/step]	q [veh/step, total]	Regime	Comment
Light traffic	0.05	0.3	1.70	0.17	Free flow	no jams
Moderate	0.10	0.3	1.70	0.34	Free flow	rare lane changes
Near-critical	0.20	0.3	1.70	0.68	Free flow	sensitive to noise
Critical	0.29	0.3	1.70	0.99	$q_{\max}$	spontaneous jams
Congested	0.50	0.3	0.70	0.70	Congested	wide jams, $c_{\text{jam}} < 0$
Rain (moderate)	0.10	0.7	1.30	0.26	Free flow	−24% flow
Rain (critical)	0.19	0.7	1.30	0.49	$q_{\max}$	−50% vs. dry
Bottleneck (acc.)	0.29	0.3	1.20	0.66	Bottleneck	$\eta \approx 0.66$

Table 3: Analytical-simulation summary across representative operating points. Total flow is summed over the two main lanes; the bottleneck row corresponds to a triggered accident in the right lane at cell 65.

4.7 Macroscopic Phenomenology: Synthesis

Bringing the above analyses together, the following compact statements summarise the macroscopic behaviour of the simulator:

Remark 1 (Free-flow throughput). *For $\rho < \rho_c \approx 0.29$, the per-lane throughput grows linearly with density at slope $v_{\max} - p$. This branch is robust to moderate ramp inflow and is essentially insensitive to the lane-change rule.*

Remark 2 (Capacity drop). *The on-ramp imposes a structural capacity drop of approximately 25–35% on the right main lane. The drop is asymmetric: the left lane is impacted only via the lane-change incentive, with a delay roughly equal to the time required for the right-lane queue to saturate.*

Remark 3 (Jam-wave kinematics). *Once nucleated, jam fronts propagate upstream at $c_{\text{jam}} \approx -1 \text{ cell/step} \approx -27 \text{ km/h}$, independently of the position of the original perturbation. This universality is a hallmark of the NaSch model and a strong indicator of the implementation’s fidelity.*

Remark 4 (Weather-induced degradation). *Doubling the randomisation probability (the “rain” control) reduces the free-flow speed by approximately 24% and the maximum flow by approximately 40%. The disproportionate sensitivity of $q_{\max}$ to p confirms the non-linear coupling between microscopic noise and macroscopic capacity.*

5 Conclusions

This report has presented a comprehensive academic evaluation of **TrafficJamEngine**, a two-lane Nagel–Schreckenberg cellular automaton with on-ramp coupling, exposed through a real-time HTTP API. The evaluation spanned three complementary dimensions: implementation fidelity, algorithmic complexity, and macroscopic phenomenology derived through analytical simulation.

On implementation fidelity. The simulation kernel implements all four canonical NaSch rules correctly, with appropriate fusion of acceleration and braking into a single non-decreasing operator and with a synchronous lane-change phase that avoids the well-known sequential-update artefact. The asymmetric merging rule is realistic, conservative, and unambiguously encodes the no-priority-for-merging-traffic principle. We observed three modelling choices—conservative rear-gap criterion, open downstream boundary, and forced-stop ramp tail—that depart from the strictest published formulations but remain academically defensible.

On computational efficiency. The cost of one global update is $\mathcal{O}(L v_{\max})$ in time and $\Theta(L)$ in memory (Proposition 1). For the default geometry $(L, v_{\max}) = (150, 2)$ this corresponds to roughly 10^3 elementary operations per step, comfortably below the budget implied by the worker tick of 0.15 s. The implementation is therefore well-matched to interactive use, with substantial headroom for parameter studies of moderate scale.

On phenomenology. The analytical simulation reproduces the canonical inverted- Λ fundamental diagram, predicts a critical density $\rho_c \approx 0.29$ and a per-lane peak flow $q_{\max} \approx 0.50$ vehicles/cell/step, and recovers the characteristic upstream-propagating jam waves at characteristic upstream-propagating jam waves at $c_{\text{jam}} \approx -27$ km/h. The on-ramp is shown to generate a structural capacity drop of 25–35% in the right main lane and to function as a deterministic nucleator of phantom jams, in close qualitative correspondence with empirical motorway observations. Inclement-weather settings ($p = 0.7$) reduce capacity by approximately 40%, an order of magnitude consistent with field measurements.

Limitations and outlook. Three avenues for extension are particularly attractive. First, a heterogeneous fleet—differentiated $v_{\max}$ for cars, lorries, and buses—would allow study of the asymmetric impact of slow vehicles on multi-lane capacity. Second, the substitution of the discrete spawn process by a Poissonised inflow with controllable mean would render the boundary statistics more faithful to empirical traffic counts. Third, vectorising the kernel through NumPy or compiling it with Numba would unlock Monte-Carlo studies of the $(\rho, p, \rho_{\text{ramp}})$ parameter cube at sample sizes sufficient to resolve the phase boundary between free flow and synchronised flow with statistical significance.

In its present form, **TrafficJamEngine** is an exemplary pedagogical artefact: it is small enough to be read in an afternoon, faithful enough to reproduce the canonical phenomenology of motorway traffic, and interactive enough to support live experimentation. It constitutes a defensible reference implementation for a Computer Simulation course on cellular-automaton-based traffic modelling.

References

- [1] K. Nagel and M. Schreckenberg. A cellular automaton model for freeway traffic. *Journal de Physique I*, 2(12):2221–2229, 1992.
- [2] M. Rickert, K. Nagel, M. Schreckenberg, and A. Latour. Two lane traffic simulations using cellular automata. *Physica A*, 231(4):534–550, 1996.
- [3] A. Schadschneider. The Nagel–Schreckenberg model revisited. *European Physical Journal B*, 10(3):573–582, 1999.
- [4] B. S. Kerner. *The Physics of Traffic*. Springer, Berlin, 2004.
- [5] D. Helbing. Traffic and related self-driven many-particle systems. *Reviews of Modern Physics*, 73(4):1067–1141, 2001.